Progress Report #4

Project Title: Email Spam Detector: Chrome Extension with Python Backend

Course: 4495 – Applied Research Project, Section: 002

Student Name: Gunleen Kaur (300373306)

Work Logs

Date	Hours Worked	Description of Work Done
Jan 14, 2025	2	Conducted initial research on spam detection techniques, including keyword filtering, heuristic algorithms, Bayesian classifiers, and machine learning. Compiled findings into a research document
Jan 16, 2025	1.5	Researched tools and components required for Chrome extension development. Explored Chrome API documentation and feasibility of integrating with Gmail.
Jan 20, 2025	3	Defined project structure, created a preliminary framework for backend and frontend integration, and discussed Flask API implementation. Documented key design decisions.
Jan 22, 2025	0.5	Created project repository, added initial files and organized folder structure for efficient project management. Encountered difficulty in designing an optimal file structure for seamless integration between frontend and backend.
Jan 25, 2025	1.5	Finalized project proposal, incorporated feedback, and documented technical specifications. Submitted proposal for review.
Jan 27,2025	1.5	Research done on the dataset for project.
Jan 30,2025	2	Started working on project using Jupyter, with the help of Python.
Feb 01,2025	1.5	Faced difficulty in pushing folders on GitHub, tried to figure out the problem.
Feb 05,2025	2.5	Worked together as team to find out errors encountered while data cleaning
Feb 07,2025	2	Worked on pushing files using command prompt and visual studio.
Feb 08,2025	3	Created project report, analysed the work done and challenges faced in project.
Feb 09,2025	2.5	Finalised the project report and submitted for grading.
Feb 12,2025	1.5	Installation of libraries in the independent environment
Feb 17,2025	2	Started work on Front end
Feb 19,2025	3.5	Did model training and worked on front end for user using html and JavaScript
Feb 23,2025	4	Created report and made video for submission.
Feb 24, 2025	2	Researched and analyzed techniques for automatic email detection within a Chrome Extension. Explored DOM parsing and Gmail API integration.
Feb 25, 2025	2.5	Reviewed Chrome Extension security policies and identified potential limitations affecting automatic email extraction. Documented initial findings.
Feb 26, 2025	3	Implemented initial email extraction functionality using DOM parsing. Encountered issues with restricted email access due to security policies.
Feb 2, 2025	3	Tested different ways to bypass security restrictions while

		complying with Chrome policies. Attempted content script injection to read email content.
Feb 28, 2025	2.5	Attempted to integrate Gmail API for automatic email detection. Faced permission constraints and execution limitations. Documented findings.
Mar 1, 2025	2	Worked on refining the API request structure to handle incoming email data efficiently. Implemented request validation checks.
Mar 2, 2025	3	Improved spam classification model by refining preprocessing techniques for better accuracy. Tested different feature extraction methods.
Mar 3, 2025	2.5	Researched alternative classification models for spam detection. Compared traditional ML (Naïve Bayes, SVM) with deep learning-based approaches (LSTM, CNN).
Mar 4, 2025	2	Developed API endpoints to process extracted email content and return classification results. Ensured efficient request handling.
Mar 5, 2025	3	Conducted integration testing between Chrome Extension and API backend. Fixed minor request-response inconsistencies.
Mar 6, 2025	2.5	Debugged CORS issues affecting communication between the Chrome Extension and backend API. Implemented solutions to improve request-response handling.
Mar 7, 2025	3	Explored UI improvements for the extension, including a clear classification indicator for spam vs. non-spam emails. Added initial styling and layout changes.
Mar 8, 2025	3	Enhanced manual email extraction within the extension to improve user experience. Integrated API call functionality for real-time classification.
Mar 9, 2025	2.5	Conducted speed and performance tests on email classification response time. Optimized API query handling to improve response speed.
Mar 10, 2025	4	Implemented logging mechanisms to track API requests and responses for debugging purposes. Conducted initial tests on backend stability.
Mar 12, 2025	3	Conducted end-to-end testing of the extension and backend integration. Validated classification accuracy and frontend response time.
Mar 13, 2025	2.5	Created detailed documentation on the current system design, listing API endpoints, request-response format, and extension functionality.
Mar 14, 2025	4.5	Documented failed attempts at automatic email detection and explored alternative approaches for future implementation. Planned next steps, including possible use of OAuth for accessing Gmail data securely.
March 15,2025	3	Prepared progress report 2
Mar 16, 2025	2.5	Attempted to refine Gmail API authentication using OAuth but faced persistent authorization issues. Explored alternatives, but no breakthrough.

Mar 17, 2025	3	Debugged backend API inconsistencies that affected response accuracy. Minor improvements but no significant progress.
Mar 18, 2025	2.5	Refined spam classification model, tested variations of feature extraction, but accuracy improvements were minimal.
Mar 19, 2025	3	Explored an alternative way to extract emails using Chrome DevTools protocol, but implementation is complex and time-consuming.
Mar 20, 2025	2.5	Conducted further debugging of front-end and backend integration. Persistent issues with handling large email bodies efficiently.
Mar 21, 2025	3	Attempted to optimize UI responsiveness but faced challenges with rendering results dynamically.
Mar 22, 2025	4	Explored spam detection model tuning, including hyperparameter optimization, but with minimal improvements.
Mar 23, 2025	4.5	Revised project documentation to reflect current progress and failures. Tested different debugging strategies for API issues. Created and submitted project report 3
Mar 24, 2025	3	Researched XGBoost for spam classification and compared it with existing ML models. Implemented an initial version in Python.
Mar 25, 2025	3.5	Tuned hyperparameters of XGBoost using GridSearchCV and tested accuracy against Naïve Bayes and SVM models.
Mar 26, 2025	4	Integrated XGBoost into backend API, replacing the previous model. Conducted performance testing.
Mar 27, 2025	3	Researched AngularJS and started refactoring front-end logic from JS to AngularJS for better state management.
Mar 28, 2025	4	Developed AngularJS components for real-time spam classification results and tested UI responsiveness.
Mar 29, 2025	3.5	Conducted debugging on AngularJS integration and fixed API request-response issues. Improved UI responsiveness.
Mar 30, 2025	5.5	Finalized XGBoost model performance evaluation, documented findings. Created project report 4

Description of Work Done

1. XGBoost Model Implementation

- Evaluated XGBoost against Naïve Bayes, SVM, and LSTM models for spam classification.
- Achieved better accuracy and performance using optimized hyperparameters.
- Replaced previous ML model with XGBoost in the backend API.
- Conducted speed and efficiency testing, showing an improvement in classification response time.

2. AngularJS Frontend Development

- Transitioned from vanilla JavaScript to AngularJS for improved maintainability and scalability.
- Created reusable AngularJS components for spam classification results.
- Integrated backend API calls efficiently with AngularJS, reducing latency and improving user experience.
- Fixed UI rendering delays and ensured real-time updates.

3. Debugging and Performance Improvements

- Resolved API request bottlenecks affecting email classification speed.
- Improved database query handling for faster data retrieval.
- Enhanced logging and error tracking mechanisms.

Current Challenges and Next Steps

Challenges Faced:

- Initial difficulty in optimizing XGBoost hyperparameters for best accuracy.
- AngularJS integration required restructuring existing frontend logic.
- Performance issues due to increased complexity in UI interactions.

Next Steps:

- Optimize AngularJS components further for a seamless user experience.
- Implement additional spam detection metrics to improve classification confidence.
- Finalize OAuth authentication method for secure Gmail data access.

Repo Check-In of Implementation:

- **Backend:** Integrated XGBoost, improved API request handling.
- **Frontend:** Migrated to AngularJS, improved UI responsiveness.
- **Documentation:** Updated integration challenges, model performance comparison.

Conclusion

Significant progress has been made by implementing XGBoost for spam classification and transitioning to AngularJS for frontend development. While challenges were encountered in performance optimization and API integration, the project is now more scalable and efficient. Future work will focus on refining the UI, optimizing the model further.